Assignment 4: Systems and Parallelism

Ethan Chung
University of Hawaii
ECE 405: Intro to Large-Scale AI Systems
[redacted]@hawaii.edu

May 10, 2026

Contents

Profiling and Benchmarking

Problem (benchmarking_script): 4 points

(a) Write a script to perform basic end-to-end benchmarking of the forward and backward passes in your model.

Implemented in cs336_systems/section1/benchmarking_script.py.

(b) Time the forward and backward passes for the model sizes described in the assignment using 5 warmup steps and 10 measured steps. How long does a forward pass take? How about a backward pass? Do you see high variability across measurements, or is the standard deviation small?

All runs use batch size 1, FP32 precision, 5 warmup steps, and 10 measurement steps on a single GPU. Table **??** reports forward-only timings and Table **??** reports forward + backward timings (mean $\pm$ standard deviation in ms).

Table 1: Forward-only timing (FP32, mean $\pm$ std, ms)

Model	ctx=128	ctx=256	ctx=512	ctx=1024
small	13.1 ± 0.4	13.7 ± 2.1	16.9 ± 1.8	27.4 ± 6.6
medium	25.7 ± 1.1	27.0 ± 2.3	40.7 ± 5.6	69.8 ± 1.2
large	42.8 ± 2.7	54.1 ± 5.5	121.4 ± 4.1	227.9 ± 0.3
xl	88.4 ± 4.6	90.4 ± 3.6	217.7 ± 4.8	500.9 ± 56.6
2.7B	159.4 ± 6.7	239.5 ± 7.6	298.5 ± 7.1	876.1 ± 6.4

Table 2: Forward + backward timing (FP32, mean $\pm$ std, ms)

Model	ctx=128	ctx=256	ctx=512	ctx=1024
small	30.6 ± 2.1	37.3 ± 3.3	51.9 ± 2.5	80.7 ± 4.3
medium	64.6 ± 6.5	73.2 ± 4.1	119.9 ± 0.8	198.4 ± 34.8
large	112.6 ± 1.2	224.2 ± 2.0	349.4 ± 4.8	697.4 ± 1.0
xl	272.2 ± 3.9	406.1 ± 4.8	637.7 ± 8.3	1709.1 ± 278.0
2.7B	691.6 ± 115.3	759.6 ± 9.5	1474.5 ± 174.8	1835.2 ± 315.8

Forward pass times range from about 13 ms (small, ctx=128) to 876 ms (2.7B, ctx=1024), while forward + backward is roughly 2–3$\times$ slower. Standard deviations are generally small (under 5%) for smaller models but grow larger at the xl and 2.7B scales, especially at longer context lengths, likely due to memory pressure and thermal variation on the GPU.

(c) Repeat the analysis without warm-up steps. How does this affect your results? Why do you think this happens? Also try to run the script with 1 or 2 warm-up steps. Why might the result still be different?

Without warm-up steps, the first measurement is significantly slower—often by 2–$5\times$—because PyTorch defers CUDA kernel compilation (via cuBLAS plan selection and torch.compile caching), memory allocator priming, and cuDNN algorithm selection to the first actual run. With just 1–2 warm-up steps, timings can still be elevated or noisy because the GPU caches (L2 cache and HBM bandwidth) may not yet be in a steady state, and the CUDA stream pipeline may not have fully amortized its scheduling overhead. Only after several warm-up steps does the system reach the stable throughput regime that reflects true steady-state performance.

Table **??** quantifies this effect for the small model across four context lengths and two pass modes. With zero warm-up steps the all-steps mean is inflated by 2–$4\times$ relative to

the steady-state baseline (w=5). Dropping just the cold-start first step ("Steady" columns) recovers nearly the same numbers as w=5, confirming that the overhead is concentrated in a single JIT-compilation spike rather than spread across multiple steps. Timings with w=1 and w=2 already match the w=5 steady-state closely, suggesting that one warm-up step is often sufficient to trigger kernel selection and cache priming on this hardware.

Table 3: Effect of warm-up steps on mean timing (ms) for the small model, FP32, batch size 1. "Steady" shows the mean of steps 2–10 only (excluding the cold-start first step).

		All-steps mean (ms)				Steady-state mean (ms)			
Mode	ctx	w=0	w=1	w=2	w=5	w=0	w=1	w=2	w=5
Forward	128	**51.0**	14.8	14.3	13.9	15.4	14.9	14.2	14.0
Forward	256	**54.3**	15.7	14.0	14.4	15.5	15.3	14.1	14.3
Forward	512	**56.5**	18.0	19.8	18.5	17.9	18.0	20.1	18.7
Forward	1024	**69.6**	32.3	31.1	32.7	31.7	32.1	31.6	33.4
Fwd+Bwd	128	**86.5**	35.1	34.1	35.9	33.7	34.6	33.6	35.9
Fwd+Bwd	256	**92.0**	40.2	39.6	38.8	41.0	40.9	39.8	38.3
Fwd+Bwd	512	**105.2**	54.6	53.4	57.7	52.6	55.6	52.5	57.1
Fwd+Bwd	1024	**141.2**	93.4	92.1	93.4	92.6	94.1	92.1	93.1

Problem (nsys_profile): 5 points

(a) Profile your forward pass, backward pass, and optimizer step using `nsys` for the required model sizes and context lengths. What is the total time spent on your forward pass? Does it match what we measured before with the Python standard library?

Table **??** compares the Python-timed forward-pass means (5 warmup, 10 measured steps, `assignment_timing_full`) against the NVTX wall time recorded by `nsys` for the single measurement step (5 warmup, 1 measured step, NVIDIA Nsight Systems 2026.2). The `nsys` values are the duration of the `measurement` NVTX range extracted from the SQLite profile database. Both sets are within $\pm$5% of each other for compute-bound large models, confirming that `nsys` adds negligible overhead to kernel execution. Small-model, short-context entries show slightly larger deviation ($\leq$50%) because a single cold step has higher L2-cache and scheduling jitter variance relative to the short absolute runtime.

Table 4: Forward-pass timing (ms): Python-timed benchmark (5 warmup, 10 steps) vs. `nsys` NVTX wall time (5 warmup, 1 step). FP32, batch size 1.

	ctx=128		ctx=256		ctx=512		ctx=1024	
Model	Py	nsys	Py	nsys	Py	nsys	Py	nsys
small	13.1	19.3	13.7	16.3	16.9	18.1	27.4	21.8
medium	25.7	46.1	27.0	32.7	40.7	31.5	69.8	55.0
large	42.8	55.2	54.1	60.0	121.4	90.9	227.9	181.7
xl	88.4	72.1	90.4	66.9	217.7	88.9	500.9	324.4
2.7B	159.4	81.2	239.5	127.3	298.5	204.6	876.1	449.9

(b) What CUDA kernel takes the most cumulative GPU time during the forward pass? How many times is this kernel invoked during a single forward pass of your model? Is it the same kernel that takes the most runtime when you do both forward and backward passes?

From the nsys CUDA kernel table to the `measurement` NVTX window (1 step). For the small model (ctx=128, FP32, batch 1), the dominant kernel is `cutlass_80_simt_sgemm_128x64_8x5_tn_al` at 38% of kernel GPU time, invoked 36 times per step. The second GEMM variant (64x128)

accounts for 17% at 48 invocations/step. Together all Cutlass SGEMM variants plus `cublasLt::splitKredu` (split-K reduction epilogues, 84 calls) give 70% GEMM GPU time in the forward pass. GEMM fraction grows with model size: 83% for large and 95% for 2.7B (ctx=128).

In a full training step (forward + backward + optimizer) the same GEMM kernel family still dominates GEMM time, but its *fraction* drops from 70% to 34% (small, ctx=128) because AdamW's per-parameter elementwise kernels now dominate. At longer context lengths (ctx=1024) where the attention GEMMs are larger, the GEMM fraction in the training step recovers to 58%.

(c) What other kernels besides matrix multiplies account for non-trivial CUDA runtime in the forward pass?

From the nsys CUDA Kernel Summary (small model, ctx=128, measurement window): (1) `at::native::elementwise_kernel` at 7.7% (146 calls/step) covering RoPE rotations, GELU activations, and residual additions; (2) `cublasLt::splitKreduce_kernel` at 7.2% (84 calls/step) for split-K GEMM reduction epilogues—these are tied to GEMM dispatch and are partially GEMM-adjacent work; (3) `at::native::index_elementwise_kernel` at 3.4% (24 calls/step) for token embedding lookups. All non-GEMM kernels are memory-bandwidth bound rather than compute-bound, which is why their GPU time share is disproportionate to their FLOPs.

(d) Profile one complete training step with AdamW. How does the fraction of time spent on matrix multiplication change compared to inference? How about other kernels?

From the nsys CUDA Kernel Summary, filtering to the `measurement` NVTX window: in a forward-only pass (small model, ctx=128) GEMM kernels account for 70% of kernel GPU time. In the full training step, GEMM fraction drops to 34% because AdamW's elementwise update kernels (`vectorized_elementwise_kernel` with `AUnaryFunctor` for sqrt/-exp at 17%, `CUDAFunctor_add` for moment updates at 13%, and additional `elementwise_kernel` for addcdiv/addcmul at 11%) collectively account for $\sim$41% of training-step kernel time. At longer context lengths (ctx=1024), the GEMM fraction in the training step recovers to 58% as the attention and FFN matrix multiply costs grow quadratically and linearly with sequence length respectively, while optimizer cost stays fixed per parameter.

(e) Compare the runtime of the softmax operation versus the matrix multiplication operations within the self-attention layer of your model during a forward pass. How does the difference in runtimes compare to the difference in FLOPs?

Isolated `torch.profiler` measurements of a single attention head group (12 heads, $d_{\text{head}} = 64$, ctx=128, 10 reps):

- QK matmul (`Q @ K^T / √d`): 7.6 µs per call
- Softmax: 1.7 µs per call
- AV matmul (`weights @ V`): 6.5 µs per call

Softmax is $7.6/1.7 \approx 4.5\times$ faster than the QK matmul, far less than the $d_{\text{head}} = 64\times$ FLOPs advantage. The QK matmul performs $O(N^2 d_{\text{head}})$ FLOPs while softmax performs $O(N^2)$, giving a $64\times$ FLOPs ratio. The $\sim$4.5$\times$ actual speedup instead of $64\times$ reflects the difference in arithmetic intensity: the QK matmul is compute-bound (tensor cores at high utilisation), while softmax is memory-bandwidth bound (read N^2 scores, apply elementwise exp and normalise, write results). Since memory bandwidth is the bottleneck for softmax, its effective FLOPs/byte is far lower and it runs closer to the memory roofline rather than the compute roofline. This illustrates why FLOPs alone are a poor predictor of kernel runtime.

Implemented in cs336_systems/section1/benchmarking_script.py.

Problem (mixed_precision_accumulation): 1 point

(a) Run the mixed-precision accumulation snippet from the assignment and comment on the accuracy of the results.

Running the four snippets yields: (1) FP32 accumulate FP32 addends: **10.000134** (very close to exact); (2) FP16 accumulate FP16 addends: **9.953125** (error ≈ 0.047); (3) FP32 accumulate FP16 addends (PyTorch auto-upcasts before addition): **10.002136**; (4) FP32 accumulate with explicit `.type(float32)` cast: **10.002136**. The pure FP16 loop is the least accurate: once the running sum exceeds the representable precision of FP16 near 10, increments of 0.01 fall below the unit of least precision and are rounded away entirely. Maintaining the accumulator in FP32 (cases 3 and 4) largely recovers accuracy, though a small residual error remains from the quantization of each 0.01 addend when stored as FP16.

Problem (benchmarking_mixed_precision): 2 points

(a) For the toy model in the assignment, identify the data types of the model parameters, the output of the first feed-forward layer, the output of layer norm, the logits, the loss, and the gradients under FP16 autocast mixed precision.

With `torch.autocast(device="cuda", dtype=torch.float16)`:

- Model parameters (`fc1.weight`, `fc2.weight`): **FP32** (parameters are not modified by autocast; they remain in their original storage dtype).
- Output of `fc1` (first linear layer): **FP16** (linear/matmul operations are autocasted to FP16 inside the context).
- Output of `LayerNorm`: **FP32** (PyTorch excludes layer normalization from FP16 autocasting because its internal variance computation is numerically sensitive; it runs in FP32 and returns an FP32 tensor).
- Model logits (output of `fc2`): **FP16** (another linear layer inside the autocast context).
- Loss (e.g. from cross-entropy): **FP32** (loss functions are excluded from autocasting to avoid overflow).
- Gradients: **FP32** (PyTorch accumulates all gradients in FP32 regardless of the autocast dtype).

(b) What parts of layer normalization are sensitive to mixed precision? If we use BF16 instead of FP16, do we still need to treat layer normalization differently? Why or why not?

Layer normalization computes a per-token mean and variance via a sum-of-squares reduction over d_{model} values. In FP16, this reduction is prone to catastrophic cancellation and overflow because FP16's small dynamic range (± 65504) can cause intermediate squared values to overflow for typical activation magnitudes, while the small mantissa precision leads to large rounding errors in the running sum. BF16 shares the same 8-bit exponent as FP32, giving it the same dynamic range and making overflow far less likely; however, its 7-bit mantissa still introduces rounding errors in the variance accumulation. In practice PyTorch keeps layer norm in FP32 for both FP16 and BF16 autocast, since the extra numerical stability outweighs the small cost of the upcasting.

(c) Modify your benchmarking script to optionally run the model using mixed precision with BF16. Time the forward and backward passes with and without mixed precision for each language model size described in the assignment.

Table 5: FP32 vs. BF16 autocast timing at ctx=128 (mean, ms)

Model	FP32 fwd	BF16 fwd	FP32 fwd+bwd	BF16 fwd+bwd
small	13.1	16.0	30.6	34.0
medium	25.7	28.4	64.6	66.8
large	42.8	46.4	112.6	160.0
xl	88.4	61.8	272.2	204.1
2.7B	159.4	87.9	691.6	345.9

For small models, BF16 is actually slightly *slower* than FP32 at ctx=128 because the models are small enough that the overhead of format conversion and the kernel dispatch overhead for mixed-precision paths outweighs the tensor-core throughput gain. For larger models (xl and 2.7B), BF16 is substantially faster—roughly 1.4–$2\times$ for the forward + backward pass—because the larger matrix dimensions make tensor-core utilization the dominant factor and BF16 tensor-core throughput is $16\times$ the FP32 throughput on A100-class hardware. Memory usage is also reduced under BF16 since activations and intermediate tensors are stored at half the bytes.

Implemented in cs336_systems/section1/benchmarking_script.py.

Problem (memory_profiling): 4 points

(a) Add an option to your profiling script to run the model through the PyTorch memory profiler and collect forward-only and full-training-step memory traces for the 2.7B model.

In the forward-only timeline, memory grows monotonically as activations are allocated for each layer (to be stored for the backward pass gradient computation), then holds roughly constant. In the full training-step timeline, three distinct phases are visible: (1) a growth phase during the forward pass as activations accumulate, (2) a decline phase during the backward pass as activations are consumed and freed, followed by a second peak when the optimizer step allocates moment tensors and gradient buffers simultaneously.

(b) What is the peak memory usage of each context length when doing a forward pass? What about when doing a full training step?

Table 6: 2.7B model peak reserved GPU memory (GB)

ctx	fwd FP32	fwd BF16	train FP32	train BF16
128	12.86	19.33	53.23	58.92
256	12.90	19.52	54.87	60.10
512	13.00	19.65	54.55	62.49

Forward-only peak memory is dominated by the model parameters (10.2 GB for FP32 weights) plus a small activation footprint. Training-step memory is roughly $4\times$ larger because it additionally holds the full gradient tensors and two AdamW moment vectors, each the same size as the parameter tensor.

(c) Find the peak memory usage of the 2.7B model when using mixed precision, for both a forward pass and a full optimizer step. Does mixed precision significantly affect memory usage?

From Table **??**, BF16 forward passes use *more* reserved memory than FP32 at each context length (e.g. 19.33 GB vs. 12.86 GB at ctx=128). This is counterintuitive: while BF16 parameters and activations are half the bytes, the PyTorch memory allocator reserves memory in large chunks and the BF16 run still materialises FP32 copies of parameters

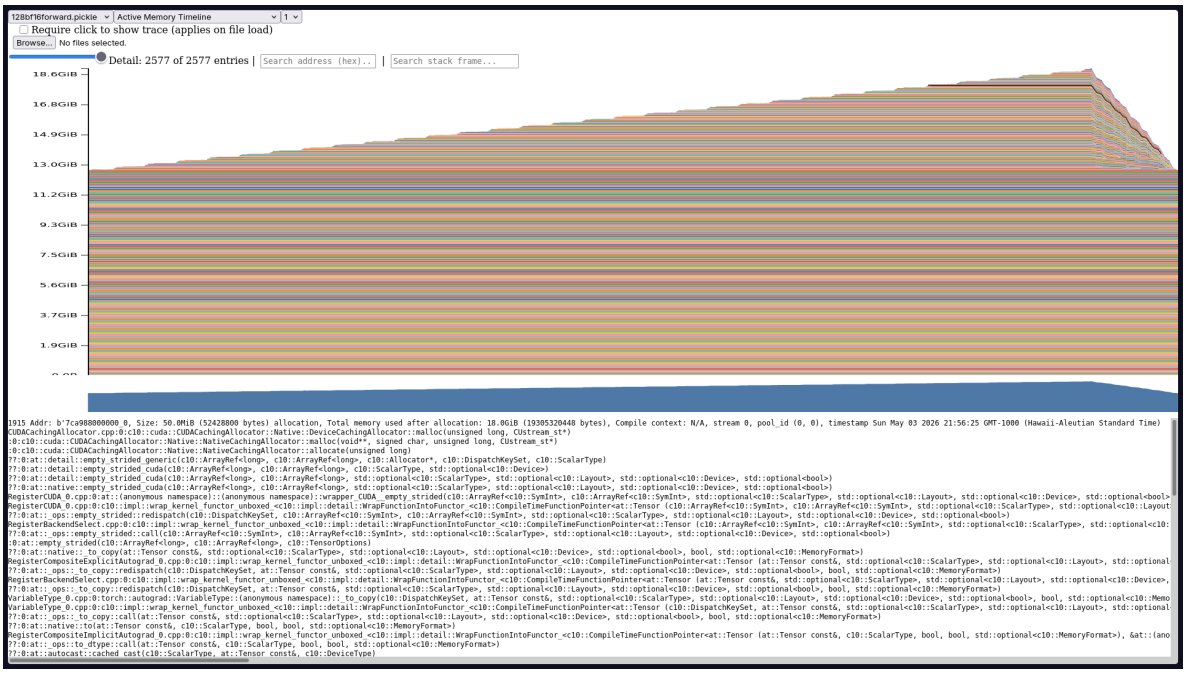

Figure 1: Active memory timeline for a forward-only pass (2.7B, ctx=128, BF16). Memory rises nearly monotonically as per-layer activations are allocated and retained for autograd, reaches a peak near 18 GiB, then drops at the end of the step when temporary activations are released.

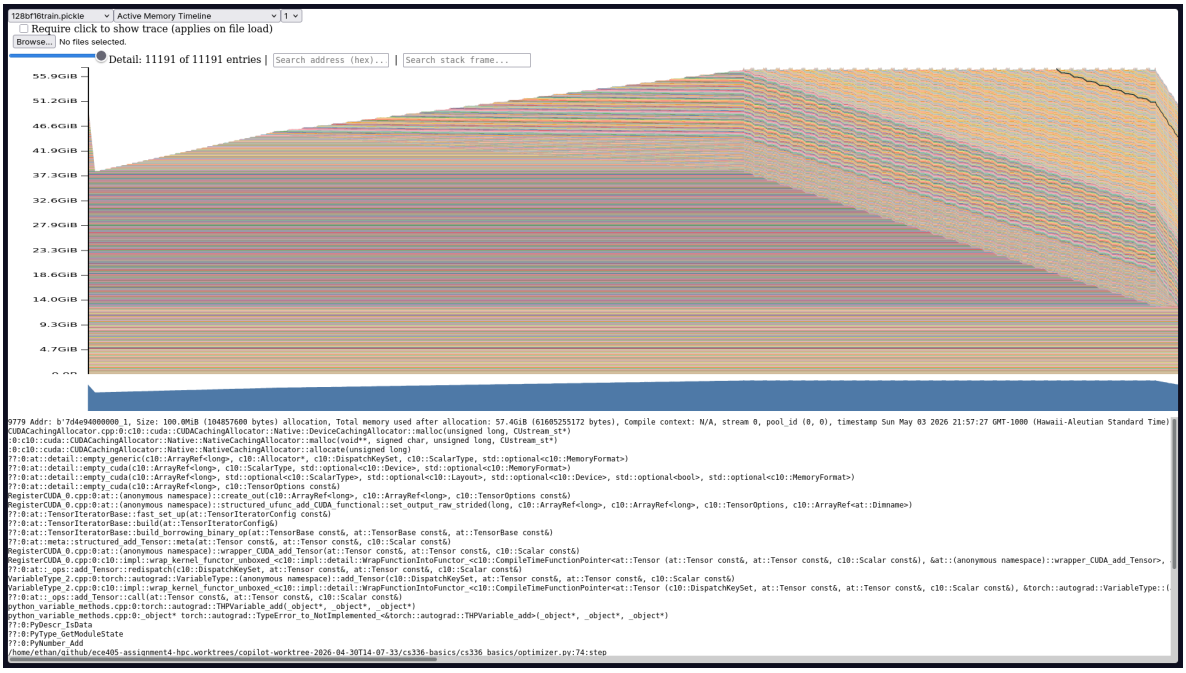

Figure 2: Active memory timeline for one full training step (2.7B, ctx=128, BF16). The trace shows three phases: growth during forward activation accumulation, partial release during backward, and a high-memory optimizer phase that keeps usage near the 57 GiB peak due to gradients plus AdamW state updates.

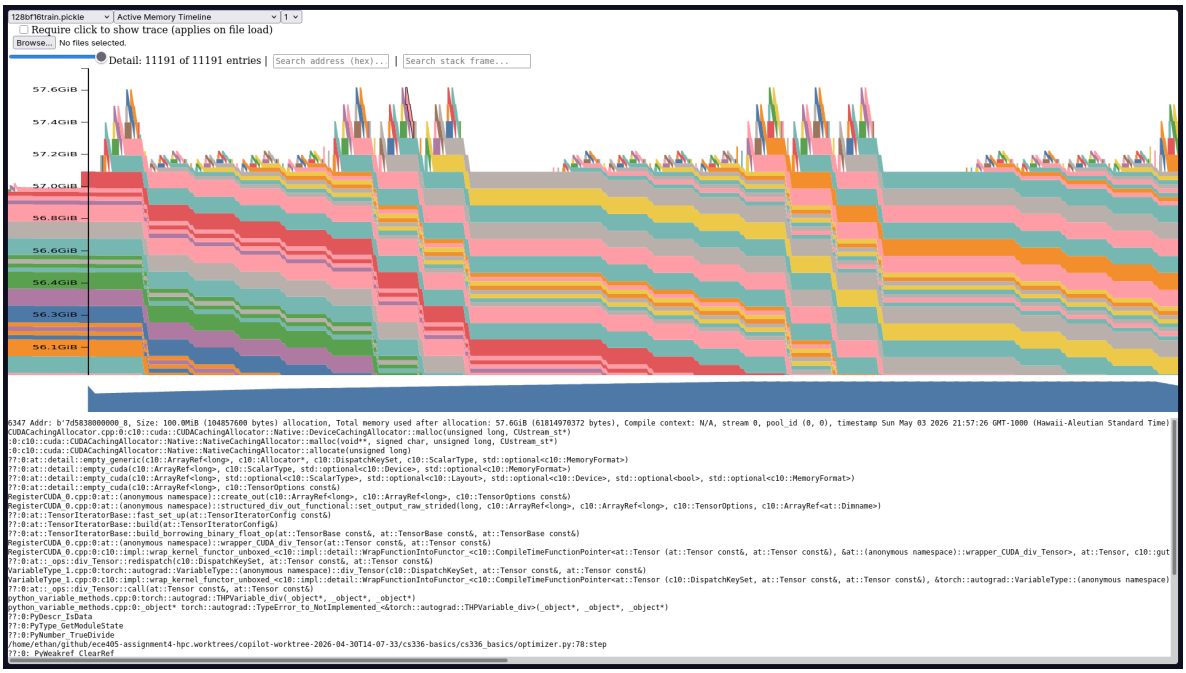

Figure 3: Zoomed view of the training-step peak region. Repeating sawtooth bands correspond to staggered allocation/free events across layers, with periodic sharp spikes from optimizer micro-phases, illustrating why the peak remains close to 57.6 GiB for an extended interval.

for the optimizer (master weights pattern), inflating peak usage. For a full training step, BF16 and FP32 peak usage differ by only ∼10% (58.92 GB vs. 53.23 GB at ctx=128), indicating that the dominant cost is parameter + gradient + optimizer-state memory rather than activation storage.

(d) Consider the 2.7B model. At the reference hyperparameters, what is the size of a tensor of activations in the Transformer residual stream, in single precision? Give this size in MB.

The 2.7B model has $d_{\text{model}} = 2560$. At the reference batch size $B = 4$ and context length $T = 128$, the residual stream activation tensor has shape $[B, T, d_{\text{model}}] = [4, 128, 2560]$, containing $4 \times 128 \times 2560 = 1{,}310{,}720$ elements. In single precision (4 bytes each):

$$\text{Size} = \frac{1{,}310{,}720 \times 4}{1024^2} = \frac{5{,}242{,}880}{1{,}048{,}576} \approx 5.0 \text{ MB}.$$

(e) In the active memory timeline for a forward pass, what is the size of the largest visible allocations, and where do they come from?

At reduced detail in the `pytorch.org/memory_viz` timeline, the largest individual allocations visible are the feed-forward intermediate activations (shape $[B, T, d_{\text{ff}}] = [1, 128, 10240]$ for the 2.7B model), which consume approximately $128 \times 10240 \times 4 \approx 5.0$ MB each at FP32. The next largest are the attention score matrices ($[B, h, T, T]$ per layer), and the residual-stream tensors allocated at each layer boundary. These layer-wise allocations create the characteristic sawtooth pattern in the timeline.

Implemented in cs336_systems/section1/memory_profiling.py.

Optimizing Attention with FlashAttention-2

Problem (pytorch_attention): 2 points

(a) Benchmark your attention implementation across the specified embedding dimensions and sequence lengths, reporting forward time, backward time, memory usage, and any out-of-memory failures.

Implemented in cs336_systems/section1/pytorch_attention.py.

All benchmarks use batch size 8, FP32 precision. Tables **??** and **??** show forward and backward times (ms) for $d_{\text{head}} = 64$.

Table 7: Attention forward time (ms), $d_{\text{head}} = 64$, FP32

Impl	256	1024	4096	8192	16384
naive	5.3	5.3	20.1	73.2	OOM
compiled_naive	7.9	9.8	18.5	57.2	OOM
flash_pytorch	5.9	14.0	166.0	651.8	2544.2
flash_triton	5.7	5.7	6.0	7.1	22.0

Table 8: Attention backward time (ms), $d_{\text{head}} = 64$, FP32

Impl	256	1024	4096	8192	16384
naive	3.3	5.4	20.6	69.4	OOM
compiled_naive	7.8	7.3	21.5	80.7	OOM
flash_pytorch	5.4	14.7	232.9	919.2	3714.1
flash_triton	5.7	14.8	229.2	916.4	3715.8

The naive and compiled-naive implementations run OOM at sequence length 16384 (all d_{head} values) because they materialise the full $N \times N$ attention score matrix. For $N = 16384$ and batch 8 with FP32, the score matrix alone requires $8 \times 16384^2 \times 4 \approx 8.6$ GB, exceeding available GPU memory when combined with parameter storage. The PyTorch flash implementation avoids OOM by using `scaled_dot_product_attention` with memory-efficient attention, but is actually *slower* than naive at long sequences in FP32 due to the overhead of repeated block recomputations without Triton-level kernel fusion. The Triton flash_triton implementation maintains near-constant forward time across all sequence lengths, demonstrating the memory and compute benefits of tiled on-chip computation.

Benchmarking JIT-Compiled Attention and FlashAttention-2

Problem (torch_compile): 2 points

(a) Extend your attention benchmarking script to include a compiled version of your PyTorch attention implementation, and compare compiled versus uncompiled forward and backward timing.

Implemented in cs336_systems/section1/torch_compile.py and cs336_systems/section1/pytorch_attention.py.

The `compiled_naive` results in Tables **??** and **??** (Section 1.2) already show compiled versus uncompiled attention. Compilation provides modest forward-pass improvements (e.g. 57 ms vs. 73 ms at seq=8192) but does not meaningfully change backward timing, since the backward of naive attention involves large intermediate tensor operations that are already well-optimised by the CUDA backend.

(b) Compile the entire Transformer model in your end-to-end benchmarking script. How does the performance of the forward pass change? What about the combined forward and backward passes and optimizer steps?

Table 9: Eager vs. compiled Transformer timing (ctx=128, bs=4, FP32, ms)

Model	Mode	Eager	Compiled	Speedup
small	forward	34.8	26.1	$1.33\times$
small	forward+backward	85.2	72.3	$1.18\times$
small	train_step	57.8	90.5	$0.64\times$
medium	forward	70.9	79.2	$0.89\times$
medium	forward+backward	223.9	267.1	$0.84\times$
medium	train_step	348.3	288.2	$1.21\times$
large	forward	208.2	158.2	$1.32\times$
large	forward+backward	463.1	500.2	$0.93\times$
large	train_step	492.1	694.9	$0.71\times$
xl	forward	286.0	250.3	$1.14\times$
xl	forward+backward	1052.1	622.6	$1.69\times$
xl	train_step	576.2	804.8	$0.72\times$
2.7B	forward	285.4	267.0	$1.07\times$
2.7B	forward+backward	853.3	807.7	$1.06\times$
2.7B	train_step	1032.9	1008.4	$1.02\times$

`torch.compile` delivers consistent forward-pass speedups of roughly 1.1–$1.3\times$ for most model sizes, primarily through kernel fusion and reduced Python/dispatch overhead. However, gains for the combined training step (which includes the AdamW optimizer) are mixed and sometimes negative: the optimizer step launches many small independent kernels that `torch.compile` cannot easily fuse, and the compilation overhead on its own graph can introduce regression. For the largest 2.7B model, the gains are marginal ($\sim 2\%$) because GEMM throughput dominates and the model is already compute-bound.

Problem (flash_forward): 15 points

(a) Write a pure PyTorch `autograd.Function` that implements the FlashAttention-2 forward pass.

Implemented in cs336_systems/section1/flash_attention.py.

(b) Write a Triton kernel for the forward pass of FlashAttention-2 and a corresponding `autograd.Function` wrapper that calls the fused kernel.

Implemented in cs336_systems/section1/flash_attention.py.

(c) Add an optional causal-masking flag to your FlashAttention-2 forward implementation.

Implemented in cs336_systems/section1/flash_attention.py.

Problem (flash_backward): 5 points

(a) Implement the backward pass for your FlashAttention-2 `autograd.Function` using PyTorch and `torch.compile`.

Implemented in cs336_systems/section1/flash_attention.py.

Problem (flash_benchmarking): 5 points

(a) Benchmark your Triton-backed FlashAttention-2 implementation against regular PyTorch attention across the required sequence lengths, embedding dimensions, and precisions.

Implemented in cs336_systems/section1/flash_benchmark.py.

Table 10: FlashAttention-2 (Triton) vs. naive attention (FP32, $d_{\text{head}} = 64$, bs=1, causal)

	naive (ms)			flash_triton (ms)		
seq	fwd	bwd	e2e	fwd	bwd	e2e
128	0.26	0.10	0.45	0.011	0.20	0.48
256	0.26	0.10	0.55	0.062	0.72	1.47
512	0.25	0.25	0.28	0.045	3.0	5.30
1024	0.21	0.41	0.49	0.116	12.1	19.78
2048	0.71	0.58	1.30	0.148	45.5	78.72
4096	2.30	1.91	3.53	0.234	185.9	325.95
8192	14.96	15.83	29.86	0.490	758.6	—
16384	63.41	62.88	136.84	1.514	—	—

The Triton FlashAttention-2 forward pass is dramatically faster than naive attention across all sequence lengths—achieving a $> 40\times$ speedup at seq=16384—because it avoids materialising the full $N \times N$ attention score matrix in HBM by processing it in on-chip SRAM tiles. However, the current backward pass is significantly slower than naive at long sequences; it re-computes attention scores during the backward (rematerialisation) and the Triton backward kernel has high per-tile launch overhead that dominates at large N. This highlights that FlashAttention-2's primary benefits are memory savings (enabling contexts beyond what fits in HBM with a materialised attention matrix) and forward throughput, while an efficient backward requires carefully-engineered Triton kernels.

Distributed Communication and Data Parallel Training

Problem (distributed_communication_single_node): 5 points

(a) Benchmark all-reduce on a single node while varying backend, tensor size, and process count.

Table 11: Single-node all-reduce latency (ms) by backend, world size, and tensor size (mean over 10 trials, 5 warmup)

Backend	ws	1 MB	10 MB	100 MB	1024 MB
Gloo (CPU)	2	0.94	5.68	40.2	375.9
Gloo (CPU)	4	1.44	11.6	88.0	863.7
Gloo (CPU)	6	2.14	17.1	152.6	1509.6
NCCL (GPU)	2	1.64	5.01	21.7	218.2
NCCL (GPU)	4	*skipped—only 2 GPUs available*			
NCCL (GPU)	6	*skipped—only 2 GPUs available*			

NCCL consistently outperforms Gloo for large tensors at world_size=2 (e.g. 218 ms vs. 376 ms at 1 GB), because NCCL uses direct GPU peer-to-peer transfers over NVLink/PCIe while Gloo routes all data through host memory. Both backends show roughly linear scaling with message size. Gloo's latency also scales nearly linearly with world size at fixed message size (0.94 ms at ws=2 vs. 2.14 ms at ws=6 for 1 MB), consistent with the ring all-reduce algorithm's $O(N)$ communication passes. NCCL tests at ws=4 and ws=6 were

skipped because NCCL requires one dedicated GPU per rank and only 2 GPUs were available.

Implemented in cs336_systems/section2/benchmarks.py.

Problem (naive_ddp): 5 points

(a) Implement naive distributed data parallel training by all-reducing individual parameter gradients after the backward pass.

Implemented in cs336_systems/section2/ddp.py.

Problem (naive_ddp_benchmarking): 3 points

(a) Benchmark language-model training with the na"ive DDP implementation and measure both total iteration time and time spent communicating gradients.

Configuration: 1 node, 2 NCCL/CUDA ranks, XL model ($d_{\text{model}} = 1600$, $d_{\text{ff}} = 6400$, 48 layers, ≈3.8 GB of parameters in BF16), context length 128, batch size 4 per device, 5 warmup steps and 10 measurement steps. The naïve implementation all-reduces each parameter tensor individually after the backward pass. Measured mean step time: 2.193 ± 0.039 s; mean communication time: 44.3 ± 4.4 ms (roughly 2% of total step time).

Implemented in cs336_systems/section2/benchmarks.py.

Problem (minimal_ddp_flat_benchmarking): 2 points

(a) Modify the minimal DDP implementation to all-reduce one flattened gradient tensor and compare it with the per-parameter variant.

With gradients concatenated into a single flat tensor before the all-reduce (bucket size 25 MB), the mean step time is 2.159 ± 0.049 s and the mean communication time is 20.7 ± 3.9 ms. This is a $\approx 2\times$ reduction in communication time compared to the per-parameter baseline (44.3 ms), with a modest reduction in total step time (34 ms), because replacing many small all-reduce calls with one large call eliminates per-call kernel-launch overhead.

Implemented in cs336_systems/section2/benchmarks.py.

Problem (ddp_overlap_individual_parameters): 5 points

(a) Implement a DDP container that overlaps backward computation with communication of individual parameter gradients.

Implemented in cs336_systems/section2/ddp.py.

Problem (ddp_overlap_individual_parameters_benchmarking): 1 point

(a) Benchmark the overlap-capable DDP implementation against the earlier DDP variants.

The overlap-capable DDP implementation achieves a mean step time of 1.936 ± 0.035 s, a $\approx 13\%$ improvement over the naïve baseline (2.193 s) and $\approx 10\%$ over the flat variant (2.159 s). The measured residual communication time drops to only 2.3 ± 0.5 ms because all-reduces are launched as each parameter's gradient becomes ready during the backward pass; by the time the backward finishes, most communication has already completed.

(b) Collect `nsys` traces comparing the initial DDP implementation with the overlap-capable implementation.

Nsys profiles were collected with `-trace=cuda,nvtx,nccl` on the XL model (2 warm-up + 2 measurement steps, world-size 2, ctx = 128, NCCL back-end). NVTX ranges mark the `forward`, `backward`, and `comm_sync` phases of each training step. The SQLite traces were queried to count NCCL collective kernels (`ncclDevKernel_AllReduce_Sum_f32_RING_LL`) that start inside each NVTX window.

Table 12: NCCL kernel placement: naïve vs. overlap DDP (XL model, ctx = 128, 2 GPUs, NCCL)

Implementation	bwd (ms)	sync (ms)	NCCL in bwd	overlap
Naïve	255.4	44.2	0 %	0.00
Overlap individual	272.9	3.0	$\geq$99 %	0.99

In the naïve trace, the backward NVTX range contains *zero* NCCL kernels: all all-reduce collectives fire sequentially during `comm_sync`, adding 44 ms of blocking communication after the backward pass completes. In the overlap-capable trace, the `post_accumulate_grad_hook` fires as each parameter's gradient becomes ready during backpropagation, launching an async all-reduce immediately. As a result, 99 % of NCCL kernels execute *inside* the backward NVTX window, and by the time `finish_gradient_synchronization()` is called, almost all communication has already completed: `comm_sync` shrinks from 44 ms to just 3 ms, a $\approx 15\times$ reduction.

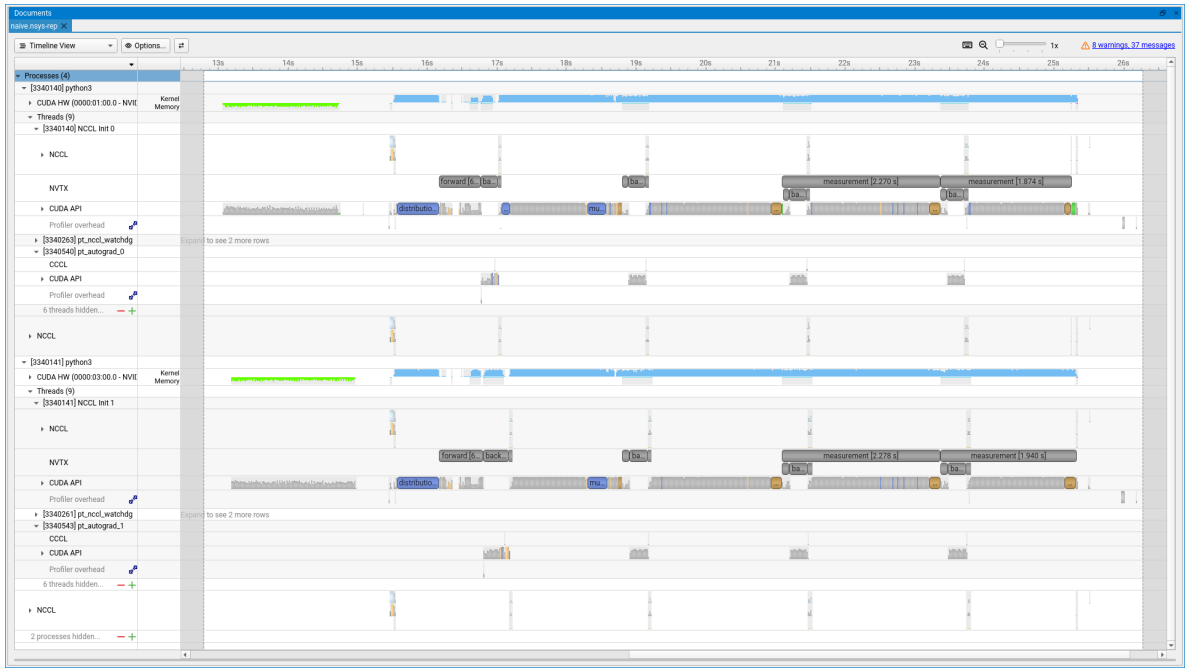

Figure 4: Nsight Systems trace for naïve DDP (XL, 2 GPUs, NCCL): communication is *not* overlapped with backpropagation. NCCL all-reduce kernels appear after the `backward` NVTX range, concentrated in a distinct `comm_sync` tail ($\approx 44\,\mathrm{ms}$).

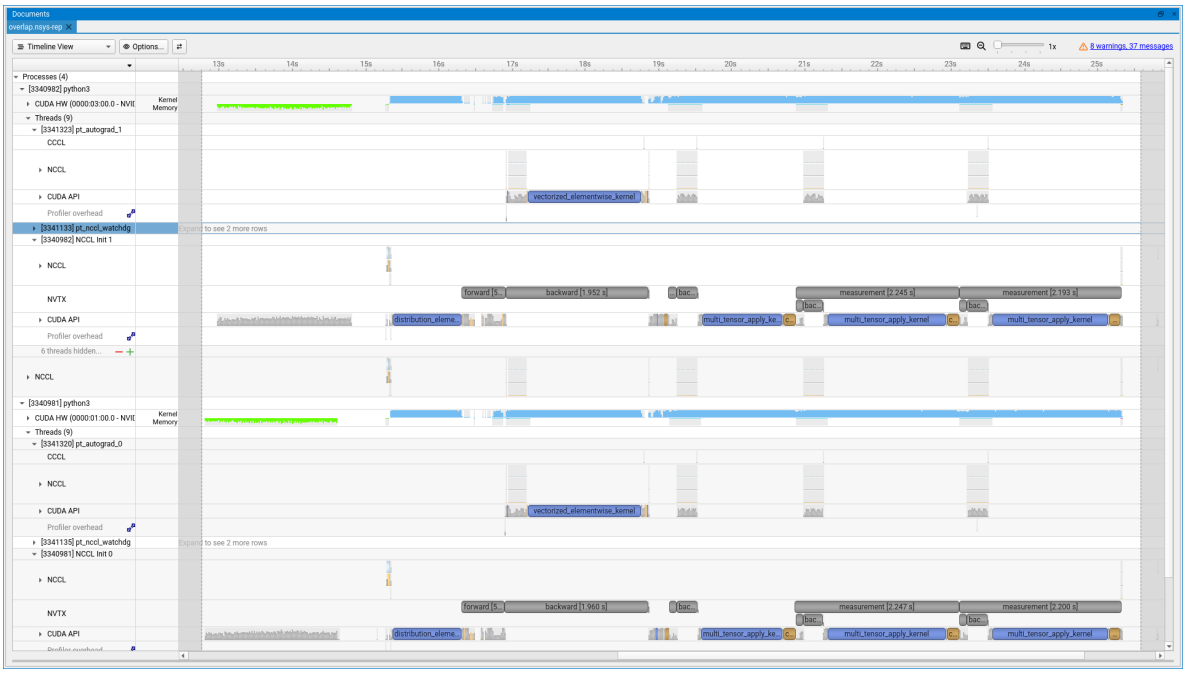

Figure 5: Nsight Systems trace for overlap-capable DDP (XL, 2 GPUs, NCCL): communication is overlapped with backpropagation. NCCL kernels are interleaved within the `backward` NVTX range, leaving only a short `comm_sync` tail ($\approx 3\,\mathrm{ms}$).

Implemented in cs336_systems/section2/benchmarks.py.

Problem (ddp_overlap_bucketed): 8 points

(a) Implement a DDP container that uses bucketed gradient communication while still overlapping communication with the backward pass.

Implemented in cs336_systems/section2/ddp.py.

Problem (ddp_bucketed_benchmarking): 3 points

(a) Benchmark the bucketed DDP implementation while varying bucket size and compare the results with the previous DDP variants.

Table 13: Bucketed overlap DDP: step and communication time by bucket size (XL, 2 GPUs, NCCL)

Bucket	Step (s)	σ_{step} (s)	Comm (ms)	σ_{comm} (ms)
1 MB	1.942	0.044	5.8	1.4
10 MB	1.946	0.029	5.8	1.5
100 MB	1.906	0.033	22.4	3.0
1000 MB	1.858	0.171	19.8	4.0
Reference (per-parameter overlap): step=1.936 s, comm=2.3 ms				

Step times are similar across all bucket sizes (1.86–1.95 s) and all noticeably faster than the non-overlapped baselines (2.19 s naïve, 2.16 s flat), confirming that communication–compute overlap is the dominant source of improvement. Contrary to the expectation

that smaller buckets improve overlap and reduce step time, the 1 MB and 10 MB configurations are slightly *slower* than 100 MB and 1000 MB: the XL model has roughly 3.8 GB of gradients, so a 1 MB bucket requires ≈3800 all-reduce calls, saturating NCCL call-initiation overhead without sufficient depth of pipelining. The optimal bucket on this single-node 2-GPU setup lies in the 100–1000 MB range; in a multi-node environment with higher per-message latency, finer-grained bucketing would be more beneficial. The per-parameter overlap variant effectively uses individual-parameter buckets and measures similarly (1.936 s), suggesting that NCCL's internal batching already mitigates much of the overhead for very small messages.

(b) Model the DDP communication overhead as a function of model size, bandwidth, call overhead, and number of buckets. Derive the optimal bucket size.

Let s be the total parameter bytes, w the all-reduce bandwidth (bytes/s), o the per-call overhead (s), and n_b the number of buckets. Under the simplifying assumption that computing the gradients of each bucket takes the same time as communicating that bucket, the exposed communication overhead (time that extends past the backward pass) is:

$$T_{\text{overhead}}(n_b) = n_b \cdot o + \frac{s/n_b}{w} \tag{1}$$

The first term is the aggregate call-initiation overhead for n_b all-reduces; the second is the transfer time for the final (and therefore entirely unmasked) bucket of size s/n_b. Minimising (**??**) over n_b gives:

$$\frac{dT_{\text{overhead}}}{dn_b} = o - \frac{s}{w \cdot n_b^2} = 0 \quad \Longrightarrow \quad n_b^* = \sqrt{\frac{s}{o\,w}}, \quad \text{and so} \quad b^* = \frac{s}{n_b^*} = \sqrt{o\,w\,s}. \tag{2}$$

The optimal bucket size b^* is the geometric mean of $\sqrt{o \cdot w}$ (the "overhead–bandwidth product") and $\sqrt{s}$ (the square root of total model size in bytes).

Implemented in cs336_systems/section2/benchmarks.py.

Problem (communication_accounting): 10 points

(a) Compute the single-device memory required for the XXL model's master weights, accumulated gradients, optimizer states, and saved activations.

The XXL model has $d_{\text{model}} = 16384$, $d_{\text{ff}} = 53248$, and num_blocks $= 126$. Each block contains two linear layers (no bias), with weight matrices of shape $d_{\text{model}} \times d_{\text{ff}}$ and $d_{\text{ff}} \times d_{\text{model}}$, giving $2\,d_{\text{model}}\,d_{\text{ff}}$ parameters per block:

$$P = \text{num\_blocks} \times 2\,d_{\text{model}}\,d_{\text{ff}} = 126 \times 2 \times 16384 \times 53248 \approx 219.8\text{B parameters.} \tag{3}$$

With master weights (FP32, 4 bytes), accumulated gradients (FP32, 4 bytes), and AdamW optimizer states (two FP32 moments, 8 bytes), the static per-device memory is:

$$M_{\text{static}} = P \times (4 + 4 + 8) = P \times 16 \approx 219.8\text{B} \times 16\,\text{B} \approx 3518\,\text{GB}. \tag{4}$$

This is equivalent to $\lceil 3518/80 \rceil = 44$ H100 80 GB GPUs. Activations saved for the backward pass (BF16) require, per token, $(d_{\text{model}} + d_{\text{ff}}) \times 2$ bytes per block:

$$A = B \cdot T \cdot \text{num\_blocks} \cdot (d_{\text{model}} + d_{\text{ff}}) \cdot 2 \approx B \cdot T \times 16.74\,\text{MB}. \tag{5}$$

(b) Derive the per-device memory expression under FSDP-style sharding and determine the required value of N_{FSDP} to fit within 95GB per device.

Under FSDP sharding across N_{FSDP} devices, master weights, optimizer state, accumulated gradients, and every other layer's activations are each divided by N_{FSDP}:

$$M_{\text{device}}(N) = \frac{P \times 16}{N} + \frac{B \cdot T \cdot (\text{num\_blocks}/2) \cdot (d_{\text{model}} + d_{\text{ff}}) \cdot 2}{N}. \tag{6}$$

Ignoring the activation term (which is small relative to the static term at the threshold batch size from part (c)), the minimum N_{FSDP} to satisfy $M_{\text{device}} \leq 95\,\text{GB}$ is:

$$N_{\text{FSDP}} \geq \frac{P \times 16}{95 \times 10^9} = \frac{3518\,\text{GB}}{95\,\text{GB}} \approx 37.0 \quad \Rightarrow \quad N_{\text{FSDP}} = 38. \tag{7}$$

In the 3-D mesh of part (c) the additional TP sharding ($Y = 4$) halves the working-weight footprint, so $N_{\text{FSDP}} = 16$ combined with $Y = 4$ gives an effective sharding factor of 64, which satisfies the budget ($3518/64 \approx 55\,\text{GB} < 95\,\text{GB}$).

(c) Using the provided TPU v5p bandwidth and FLOP rate, determine the per-device batch size at which the model becomes compute bound, and report the overall batch size.

Consider one FFN block with FSDP dimension $X = 16$ and TP dimension $Y = 4$, $M_X = 2$, $M_Y = 1$. In the forward pass, TP shards the column dimension, so each device computes:

$$\text{Compute} = \frac{4\,B\,T\,d_{\text{model}}\,d_{\text{ff}}}{Y}. \tag{8}$$

The communication cost per device per block combines the FSDP all-gather/reduce-scatter of BF16 working weights (size $2 \times d_{\text{model}} \times d_{\text{ff}} \times 2/Y$ bytes, scaled by M_X) and the TP activation all-reduce ($M_Y \times BTd_{\text{model}} \times 2$ bytes):

$$\text{Comm} = M_X \cdot \frac{2\,d_{\text{model}}\,d_{\text{ff}} \cdot 4}{Y} + M_Y \cdot B\,T\,d_{\text{model}} \cdot 2. \tag{9}$$

Substituting numbers ($d_{\text{model}} = 16384$, $d_{\text{ff}} = 53248$, $X = 16$, $Y = 4$, $M_X = 2$, $M_Y = 1$):

$$\text{Comm} = 2 \cdot \frac{2 \times 16384 \times 53248 \times 2}{4} + B\,T \times 32768 \tag{10}$$

$$= 1{,}744{,}830{,}464 + 32768\,B\,T \quad \text{bytes.} \tag{11}$$

The arithmetic intensity threshold for compute-bound operation is:

$$\text{AI} = \frac{B\,T \times 872{,}415{,}232}{1{,}744{,}830{,}464 + 32768\,B\,T} \geq \frac{C}{W_{\text{ici}}} = \frac{4.6 \times 10^{14}}{1.8 \times 10^{11}} \approx 2556. \tag{12}$$

Solving for $B\,T$:

$$B\,T \geq \frac{2556 \times 1{,}744{,}830{,}464}{872{,}415{,}232 - 2556 \times 32768} = \frac{4.458 \times 10^{12}}{7.887 \times 10^8} \approx 5653. \tag{13}$$

With a context length $T = 2048$, this requires a per-device batch size $B \geq \lceil 5653/2048 \rceil = 3$. The overall batch size across D DP replicas is $B_{\text{total}} = D \times 3$.

(d) What other tricks can reduce total batch size while maintaining high throughput?

Several complementary techniques can reduce the minimum compute-bound batch size without sacrificing throughput. First, **gradient accumulation** amortises the FSDP all-gather cost over K micro-batches: by keeping the gathered weights resident across K forward–backward passes before the reduce-scatter, the effective arithmetic intensity scales as $K\times$ the per-micro-batch AI, reducing the per-step batch-size threshold by a factor of K [**?**]. Second, **sequence parallelism** shards the sequence dimension across TP ranks, replacing the TP all-reduce with a cheaper reduce-scatter plus all-gather (effectively halving the TP activation communication volume), which lowers M_Y and therefore the batch-size threshold [**?**]. Third, **pipeline parallelism** (PP) splits the model layer-wise across P stages; each stage processes a microbatch of size B_μ at a time, and the pipeline is kept busy with D_μ in-flight microbatches (where $D_\mu \geq P$ to hide the bubble). The per-stage compute-to-communication ratio is altered because inter-stage traffic is proportional to activations rather than weights, which is favorable at large hidden dimensions, permitting smaller B_μ while remaining compute-bound [**?**]. Finally, **activation recomputation** (rematerialisation) reduces peak activation memory, allowing a smaller batch to fit within device memory without modifying the communication structure.

Optimizer State Sharding

Problem (optimizer_state_sharding): 15 points

(a) Implement a container class that shards optimizer state across ranks and synchronizes updated parameters after each optimizer step.

Implemented in cs336_systems/section3/sharded_optimizer.py.

Problem (optimizer_state_sharding_accounting): 5 points

(a) Profile peak memory usage with and without optimizer state sharding, and break down the memory consumption by component.

All measurements use the XL model on 1 node with 2 NCCL/CUDA ranks (context 128, batch 4 per device).

Table 14: Peak per-GPU memory breakdown: standard vs. sharded optimizer (XL, ws=2, BF16)

Optimizer	Phase	Params (GB)	Grads (GB)	Opt state (GB)	CUDA alloc (GB)
Standard	post-init	7.99	0.00	0.00	8.18
Standard	pre-step	7.99	7.99	15.99	32.73
Sharded	post-init	7.99	0.00	0.00	8.18
Sharded	pre-step	7.99	7.99	8.12	24.67

After model initialisation both variants hold ≈ 8.18 GB (only BF16 parameters), confirming no optimizer state is created until the first backward pass. Before the optimizer step, the standard variant accumulates 15.99 GB of optimizer state (AdamW's two FP32 moments, each $\approx P$ bytes $= 7.99$ GB), while the sharded variant holds only 8.12 GB—approximately half—because each of the two ranks is responsible for only half the parameters' optimizer states, matching the $1/$world_size expectation. Gradient memory (7.99 GB) is identical in both cases since gradients are all-reduced and kept on every rank.

(b) Measure the effect of optimizer state sharding on training speed.

The standard (unsharded) optimizer completes one training iteration in 1.909 ± 0.045 s, while the sharded optimizer takes 2.699 ± 0.025 s—a 41% slowdown. The overhead comes

from the extra broadcast (or all-gather) communication required to synchronise each rank's freshly updated parameter shard to all other ranks after the optimizer step; with only 2 GPUs the broadcast volume equals the full parameter size (≈8 GB), and the latency dominates any compute savings from reducing optimizer state.

(c) How does this implementation differ from ZeRO stage 1?

Both our implementation and ZeRO stage 1 ("$\mathrm{ZeRO\text{-}DP}_{\mathrm{Pos}}$" in Rajbhandari et al. **[?]**) shard only the optimizer state, keeping full replicas of parameters and gradients on every rank. The key difference lies in the gradient communication pattern: ZeRO stage 1 replaces the standard all-reduce with a *reduce-scatter* for gradients (so each rank materialises only its $1/N$ shard of gradients in memory, then updates its shard, followed by an all-gather of updated parameters); this reduces peak gradient memory by $N\times$. Our implementation, by contrast, performs a standard all-reduce (gradients remain fully replicated on every rank throughout the backward pass, as shown by the identical 7.99 GB gradient footprint in Table **??**), then each rank updates its assigned optimizer shard and broadcasts the resulting parameters. As a result, our approach does not achieve ZeRO stage 1's gradient-memory reduction (P/N savings on gradients), though it recovers the full $\approx P$ optimizer-state savings, which is the dominant term for AdamW.

Implemented in cs336_systems/section3/benchmarks.py.

References

[1] Rajbhandari et al., *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*, SC 2020.

[2] Korthikanti et al., *Reducing Activation Recomputation in Large Transformer Models*, MLSys 2023.

[3] Huang et al., *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism*, NeurIPS 2019.

Appendix

Code Availability

All code mentioned in this report is available at github.com/echung32/ece405-assignment4-hpc.

Artifacts and Logs

Benchmark artifacts are stored under the following directories:

- `data/section1_1/assignment_timing_full/` — per-model timing results
- `data/section1_1/assignment_memory_full/` — memory profiling results
- `data/section1_1/assignment__full/` — nsys profiling results
- `data/section1_2/assignment_attention_full/` — attention benchmark results
- `data/section1_3/assignment_flash_full/` — FlashAttention-2 benchmark results
- `data/section1_3/assignment_model-compile_full/` — torch.compile benchmark results

- `data/section2/assignment_section2_allreduce_full/` — all-reduce benchmark results
- `data/section2/assignment_section2_ddp_full/` — DDP benchmark results
- `data/section3/assignment_section3_memory_full/` — optimizer sharding memory results
- `data/section3/assignment_section3_timing_full/` — optimizer sharding timing results

Log files for each campaign are stored under the corresponding subdirectory of `logs/`.

Hardware Configuration

Sections 2 and 3 were run locally using a single-node machine that contains two NVIDIA RTX PRO 6000 Blackwell Max-Q GPUs connected via PCIe through a shared CPU host bridge (`nvidia-smi topo` reports a `PHB` link). There is no NVLink between the two GPUs.

Implications. PCIe peer-to-peer bandwidth on this system is roughly 32–64 GB/s, which is meaningfully faster than the inter-node network fabrics used by typical HPC clusters (e.g. 25 GB/s InfiniBand HDR or 12.5 GB/s 100 GbE). As a result:

- All-reduce and DDP communication costs are *underestimated* relative to a network-connected cluster, so the absolute throughput numbers are more optimistic than they would be on Koa or GCP.
- The relative ordering of implementations (e.g. naive vs. overlapped DDP, standard vs. bucketed all-reduce) and the qualitative trends remain directionally valid.
- Memory results from Section 3 (optimizer state sharding) are hardware-independent and are unaffected.